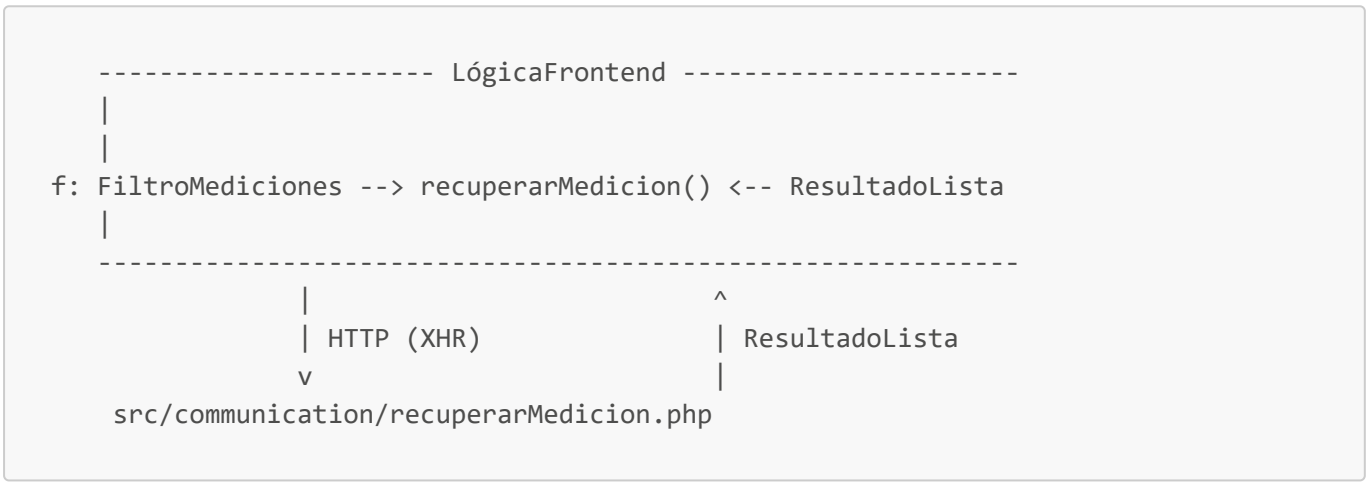
Frontend Business Logic Design (web)

Lógica de negocio del cliente web: pide las mediciones al servidor y las adapta para la pantalla. Notación: Logical Design & Reverse Engineering Specification v3.

Component Design

La lógica de negocio del frontend es el **intermediario** entre la interfaz (`gui/`) y el servidor REST (`communication/`). Expone a la interfaz **un subconjunto** de la lógica de negocio del servidor, con **firma idéntica**: la misma operación de lectura que el backend (`business_logic_design.md`), pero implementada como **proxy** (por dentro llama al servicio remoto en vez de tocar la base de datos).



Método (contrato)

Método	Tipo	Entrada	Salida
`recuperarMedicion( f )`	solo lectura (`<--`)	`FiltroMediciones`	`ResultadoLista`

`FiltroMediciones` y `ResultadoLista` son los **mismos tipos de dominio** definidos en `business_logic_design.md`; el cliente no inventa tipos nuevos.

Identidad de firma con el backend

```
BACKEND  (business_logic_design.md):
   f: FiltroMediciones --> recuperarMedicion() <-- ResultadoLista

FRONTEND (este documento):
   f: FiltroMediciones --> recuperarMedicion() <-- ResultadoLista
```

Por la **regla del subconjunto**, el cliente solo declara las operaciones que usa la interfaz (aquí, la lectura); por la **regla del proxy**, la implementación llama al endpoint `src/communication/recuperarMedicion.php` en vez de a la base de datos.

Flujo de datos (proxy)

```
   recuperarMedicion( f )
        |
        | 1. montar la dirección con las constantes de configuracion.js
        | 2. GET (XHR) a src/communication/recuperarMedicion.php
        |       -> { "mediciones": [ {id_plaquita, tipo_medicion, valor_medicion,
fecha} ] }
        | 3. adaptar los campos (id_plaquita->nombre, tipo_medicion->tipo,
valor_medicion->valor)
        |       -> ResultadoLista( ok=true, mediciones )
        | 4. (red caída / error del servidor) -> ResultadoLista( ok=false, error )
```

Configuración

`configuracion.js` es el **único punto** donde se fijan las constantes de conexión: `HOST`, `PUERTO` y `RUTA_RECUPERAR` (`"src/communication/recuperarMedicion.php"`).

Diseño de funciones (notación v3)

La lógica de negocio del frontend no define clases: es un **módulo de funciones**. La que habla con el servidor es `consultarMediciones()` (implementa, como *proxy*, la operación de dominio `recuperarMedicion()` del backend y entrega su resultado por callbacks). Las demás son **internas** (detalles de implementación). A la izquierda, `-->` apunta a la función cuando recibe datos y `<--` sale de ella cuando devuelve.

```
                ------------- frontend_business_logic -------------
                |
                |   // funciones INTERNAS (no forman parte del contrato)
                |
                |   responder( callback, valor ) --
                |   adaptarMediciones( mediciones ) --
                |   viene( objeto, campo ) --
                |   construirURL( host, puerto, ruta ) --
                |   formatearFecha( fecha ) --
                |
                |
f: FiltroMediciones --> recuperarMedicion() <-- ResultadoLista   // operación
(proxy)
                |
                |   // implementación real (asíncrona, con callbacks)
                |
callbackExito, callbackError --> consultarMediciones() -->   // llama a UNO de
ellos
                |
                -------------------------------------------------------
```

Función	Tipo	Entrada	Salida
`consultarMediciones()`	proxy (`-->`)	`callbackExito`, `callbackError`	(llama a un callback)

Función	Tipo	Entrada	Salida
`responder()`	interna (`--x`)	`callback`, `valor`	—
`adaptarMediciones()`	interna (`--x`)	`[ Medicion ] Server`	`[ Medicion ] Vista`
`viene()`	interna (`--x`)	`objeto: Medicion, campo: Text`	`B`
`construirURL()`	interna (`--x`)	`host: Text, puerto: N, ruta: Text`	`Text`
`formatearFecha()`	interna (`--x`)	`fecha: Text`	`Text`

Design Clarifications

- El frontend expone **solo lectura**: no hay `guardarMedicion()` en el cliente, porque la subida de mediciones la hace la app Android.
- `recuperarMedicion( f )` encapsula toda la comunicación: es el **único** método del contrato y el único que usa `XMLHttpRequest`. La interfaz (`gui/`) nunca habla con la red.
- Las funciones `construirURL()`, `adaptarMediciones()` y `formatearFecha()` son **detalles de implementación**, no forman parte del contrato (igual que las funciones auxiliares del backend). Son puras, no hacen red y se pueden probar con Node sin servidor.
- El resultado se entrega a la interfaz con los **mismos campos de dominio** del backend (`ok`, `mediciones`, `error`, `clase`); renombrar los campos de la tabla concreta es un detalle interno del proxy.